

UNIVERSIDAD TECNOLÓGICA NACIONAL

Trabajo Práctico Integrador – Programación 2

Alumnos:

- JEREMIAS SANTIAGO GARMENDIA MARCHESE - COMISIÓN 3 - LEGAJO 6247
- PABLO VERA - COMISIÓN 13 - LEGAJO 424033
- ROCIO SANTARELLI - COMISIÓN 7 - LEGAJO 4819

Materia:

Programación 2

Profesores:

Cinthia Rigoni y Ariel Enferrel

Tutores:

Emmanuel Avellaneda

Fecha de entrega:

25/06/2026

Índice

1. Introducción	3
2. Diseño	3
2.1. Main	3
2.2. Clases de entidad	3
2.3. Control mediante Enumeraciones	4
2.4. Interface (Calculable)	4
2.5. Errores y excepciones	5
2.6. Servicios, encapsulamiento y memoria	5
2.7. README	5
3. Desafío Técnico: Validación de datos	6
4. Conclusión y capturas	6
5. Enlaces	13
6. Bibliografía	13

1. Introducción

En el presente trabajo se pretende presentar un programa desarrollado en JAVA (versión 21) en el cual se presente todos los conocimientos brindados en el la asignatura "Programación 2".

Se presenta un programa el cual simula un sistema de gestión de pedidos de comida operado desde consola, dicho programa fue diseñado bajo el paradigma orientado a objetos aplicando conceptos como polimorfismo, herencia y una arquitectura estructurada por paquetes.

2. Diseño

2.1. Main

El programa comienza y se ejecuta desde "Main.java" el cual representa la entrada a la aplicación, en este bloque instanciamos los servicios que persistirán en memoria volátil los cuales son `CategoriaService`, `ProductoService`, `UsuarioService` y `PedidoService`, también inicializa un cargador automatizado de datos (`DataLoader`) y una interfaz de usuario (`Appmenu`) permitiendo el arranque correcto del programa.

2.2. Clases de entidad

Dentro de "entities" tendremos las diferentes clases de entidad que persistirán en memoria durante la ejecución del programa. Para este gestor de comida, se desarrollaron las siguientes clases de entidad:

- Base (Superclase): clase abstracta de la cual heredan todas las entidades del sistema utilizando el atributo "id" el cual unifica las identificaciones para el resto.
- Usuario: Define a los miembros capaces de utilizar el sistema, conformados con atributos como "nombre", "email", "password", "celular", hereda "id" de la superclase base y un rol (usuario o administrador).

- Categoría: modela las agrupaciones de “productos” mediante “nombre” y “descripción”, establece una relación 1:N con “productos” ya que en una misma categoría pueden entrar varios “productos”.
- Producto representa los artículos comerciales del inventario, posee atributos “nombre”, “precio”, “stock” y la referencia a “Categoría”, relación previamente mencionada.
- Pedido: Con cada compra, trabaja con los datos de la misma heredando Id de la clase Base y referenciando al usuario, la lista de la orden de los ítems que se encuentra en “DetallePedido”, atribuyendo un atributo “forma de pago” junto con un “estado” para controlar el ciclo de vida de la orden, también establece la fecha del momento de realizarse y utiliza el método “calcularTotal” heredado de la interfaz “Calculable” de la cual se hablará más tarde.
- DetallePedido: Trabaja ítem a ítem del “pedido” padre, referencia a “producto” y un entero “cantidad”, también un cálculo en local “calcularSubtotal” el cual multiplica el valor de “Producto” por “cantidad” para entregar un subtotal de la línea del ítem.

2.3. Control mediante Enumeraciones

Para mantener la consistencia con los datos que se trabaja, formulamos “enums”, tipos de datos especiales y fijos para algunos atributos:

- Rol: para el atributo “rol” en usuario, se formularon los valores “ADMINISTRADOR” y “CLIENTE”
- Estado: utilizado en “Pedido”, se formularon los estados “PENDIENTE”, “APROBADO”, “EN_CAMINO”, “ENTREGADO”, “CANCELADO” los cuales son los distintos pasos del ciclo de vida del pedido.
- FormaPago: utilizado en “Pedido” para los medios de pago, usando “EFECTIVO” “TARJETA” y “TRANSFERENCIA” como tipo de atributo

2.4. Interface (Calculable)

La interfaz es el contrato formal de comportamiento que obligan a las clases derivadas a proveer una implementación concreta de los métodos abstractos declarados, en este caso la interfaz central es “Calculable” la cual define el contrato formal y la firma

abstracta del método “calcularTotal” que será quien tenga la lógica financiera en este sistema.

2.5. Errores y excepciones

Para este programa se desarrolló un sistema de excepciones personalizadas propias para una mejor comunicación con el usuario ante errores de tipeo o ingreso de datos, dando un mensaje más descriptivo en interfaz, y evitar crasheos por parte del programa.

- `EntidadNoEncontradaException`: al ingresar un id para búsqueda, eliminación o edición, el cual es invalido, se dispara esta excepción, capturando la falla y advirtiendo al usuario y anulando cualquier tipo de edición durante el desarrollo de la opción elegida.
- `StockInsuficienteException`: se dispara al querer superar el stock posible frente al stock presente de un item, protegiendo así de presencia de números negativos en los items

2.6. Servicios, encapsulamiento y memoria

Para evitar que la interfaz de usuario interactúe directamente con las entidades del modelo, se creó una capa de servicios las cuales interactúan de intermediario, encapsulando las reglas de negocio y controlando el ciclo de vida completo de los datos. Los componentes desarrollados fueron:

- `UsuarioService`: Administra la persistencia de las cuentas del sistema.
- `ProductoService`: Métodos de alta y edición de items del menú, actualizando las referencias de las categorías y verificando disponibilidad de stock.
- `CategoriaService`: Procesa las altas, modificaciones de descripción y bajas de categorías de comida, interactuando con el servicio de producto.
- `PedidoService`: Maneja el flujo de pedidos, eliminar, actualizar existentes y mostrar totales.

2.7. README

Instructivo propio para la ejecución inicial del programa por parte del futuro usuario del sistema.

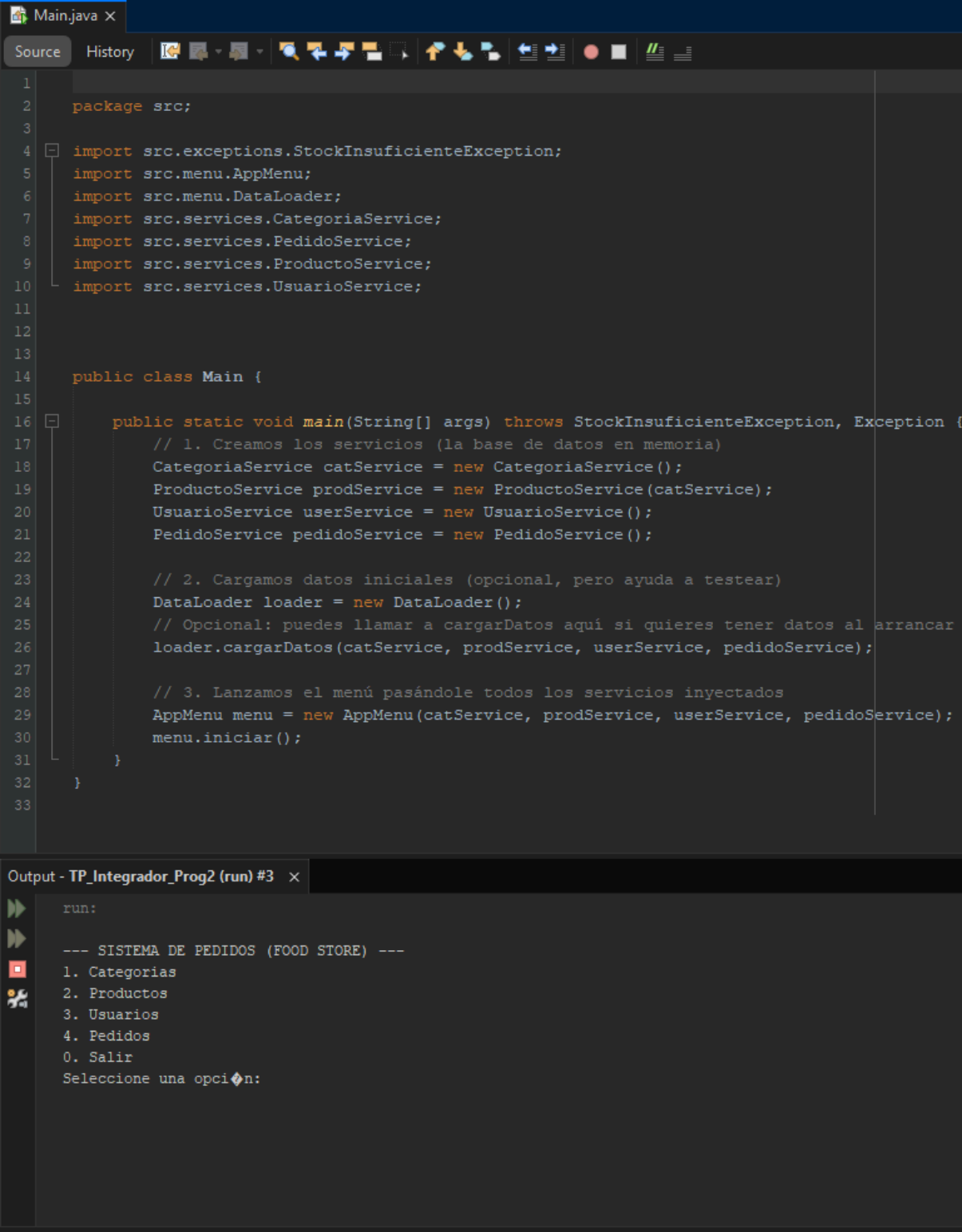
3. Desafío Técnico: Validación de datos

En la etapa de verificación del sistema, se determinó que los fallos de escritura o la ausencia de información representaban vulnerabilidades críticas para la estabilidad. Con el fin de mitigar cierres inesperados durante la ejecución de la aplicación debido a parámetros incorrectos, se procedió a una reestructuración de la lógica en la interfaz *AppMenu*. Dicha modificación consistió en asegurar los procesos vitales de creación y actualización de registros mediante el empleo de bloques *try-catch*.

Esta metodología posibilita la captura gestionada de la excepción *IllegalArgumentException* generada por la lógica de negocio. Al detectar una anomalía, el bloque de captura actúa de forma instantánea para encapsular el error, notificando al usuario mediante un mensaje legible en consola sobre la falla puntual y retornando la ejecución hacia el menú de inicio de forma segura, garantizando así la persistencia y robustez del software.

4. Conclusión y capturas

Este proyecto no solo permitió implementar todos los conocimientos vistos en programación II con respecto a la programación JAVA y la programación orientada a objetos, sino que trajo varios desafíos para superar pensando de otra manera el manejo del usuario con los programas que se vayan a realizar en el futuro.



The screenshot shows an IDE with a Java file named `Main.java`. The code defines a `package src;` and imports several classes from the `src` package: `exceptions.StockInsuficienteException`, `menu.AppMenu`, `menu.DataLoader`, `services.CategoriaService`, `services.PedidoService`, `services.ProductoService`, and `services.UsuarioService`. A `Main` class is defined with a `main` method that performs three steps: 1. Creates services (in-memory database), 2. Loads initial data (optional), and 3. Launches the menu with all services. The output window shows the program running and displaying a menu for the 'SISTEMA DE PEDIDOS (FOOD STORE)' with options: 1. Categorías, 2. Productos, 3. Usuarios, 4. Pedidos, and 0. Salir. It prompts the user to 'Seleccione una opción:'.

```
1 package src;
2
3
4 import src.exceptions.StockInsuficienteException;
5 import src.menu.AppMenu;
6 import src.menu.DataLoader;
7 import src.services.CategoriaService;
8 import src.services.PedidoService;
9 import src.services.ProductoService;
10 import src.services.UsuarioService;
11
12
13
14 public class Main {
15
16     public static void main(String[] args) throws StockInsuficienteException, Exception {
17         // 1. Creamos los servicios (la base de datos en memoria)
18         CategoriaService catService = new CategoriaService();
19         ProductoService prodService = new ProductoService(catService);
20         UsuarioService userService = new UsuarioService();
21         PedidoService pedidoService = new PedidoService();
22
23         // 2. Cargamos datos iniciales (opcional, pero ayuda a testear)
24         DataLoader loader = new DataLoader();
25         // Opcional: puedes llamar a cargarDatos aquí si quieres tener datos al arrancar
26         loader.cargarDatos(catService, prodService, userService, pedidoService);
27
28         // 3. Lanzamos el menú pasándole todos los servicios inyectados
29         AppMenu menu = new AppMenu(catService, prodService, userService, pedidoService);
30         menu.iniciar();
31     }
32 }
33
```

Output - TP_Integrador_Prog2 (run) #3 x

run:

--- SISTEMA DE PEDIDOS (FOOD STORE) ---

1. Categorías

2. Productos

3. Usuarios

4. Pedidos

0. Salir

Seleccione una opción:

```

19
20
21 public class AppMenu {
22     private Scanner scanner = new Scanner(System.in);
23     private CategoriaService categoriaService;
24     private ProductoService productoService;
25     private UsuarioService usuarioService;
26     private PedidoService pedidoService;
27
28     // Constructor
29     public AppMenu(CategoriaService cs, ProductoService prods, UsuarioService us, PedidoService ps) {
30         this.categoriaService = cs;
31         this.productoService = prods;
32         this.usuarioService = us;
33         this.pedidoService = ps;
34     }
35
36     public void iniciar() throws EntidadNoEncontradaException, Exception {
37         boolean activo = true;
38         while (activo) {
39             System.out.println("\n--- SISTEMA DE PEDIDOS (FOOD STORE) ---");
40             System.out.println("1. Categorías");
41             System.out.println("2. Productos");
42             System.out.println("3. Usuarios");
43             System.out.println("4. Pedidos");
44             System.out.println("0. Salir");
45             System.out.print("Seleccione una opción: ");
46
47             String opcion = scanner.nextLine();
48
49             // Utilizamos switch moderno para mayor legibilidad y evitar breaks
50             switch (opcion) {
51                 case "1" -> menuCategorias(); // Llamas al submenú Categorías
52                 case "2" -> menuProductos(); // Llamas al submenú Productos
53                 case "3" -> menuUsuarios(); // Llamas al submenú Usuarios

```

Output - TP_Integrador_Prog2 (run) #3

```

--- SISTEMA DE PEDIDOS (FOOD STORE) ---
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Salir

```



```
// --- SUBMENUS ---
private void menuCategorias() {
    System.out.println("\n--- GESTIÓN DE CATEGORÍAS ---");
    System.out.println("1. Listar\n2. Crear\n3. Editar \n4. Eliminar\n0. Salir");
    String op = scanner.nextLine();
    switch (op) {
        case "1" -> categoriaService.listar().forEach(System.out::println);
        case "2" -> {
            System.out.println("--- NUEVA CATEGORIA ---");
            try{
                System.out.print("Nombre: ");
                String nombre = scanner.nextLine();
                System.out.print("Descripcion: ");
                String desc = scanner.nextLine();

                // Generamos un ID automático basado en cuántos elementos hay
                Long nuevoId = (long) categoriaService.listar().size() + 1;

                // Creamos el objeto
                Categoria nueva = new Categoria(nuevoId, nombre, desc);

                // Lo guardamos en el servicio
                categoriaService.crear(nueva);
                System.out.println("¡Categoria creada correctamente!");
            }catch (IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
        case "3" -> {
            categoriaService.listar().forEach(c -> System.out.println(c.resumen()));
            try {
                System.out.print("Ingrese el ID: ");
                Long id = Long.parseLong(scanner.nextLine());
                System.out.print("Ingrese el nuevo nombre: ");
                String nuevoNombre = scanner.nextLine();
                System.out.print("Ingrese la nueva descripcion: ");
            }
        }
    }
}
```

```
private void menuProductos() throws EntidadNoEncontradaException {
    System.out.println("\n--- GESTIÓN DE PRODUCTOS ---");
    System.out.println("1. Listar\n2. Crear\n3. Editar \n4. Eliminar\n0. Salir");
    String op = scanner.nextLine();
    switch (op) {
        case "1" -> productoService.listar().forEach(System.out::println);
        case "2" -> {
            try {
                System.out.print("Nombre: ");
                String nombre = scanner.nextLine();
                System.out.print("Precio: ");
                Double precio = Double.parseDouble(scanner.nextLine());
                System.out.print("Stock: ");
                int stock = Integer.parseInt(scanner.nextLine());
                System.out.println("Categorías disponibles:");
                categoriaService.listar().forEach(System.out::println);
                System.out.print("Ingrese el ID de la categoría: ");
                Long categoriaId = Long.parseLong(scanner.nextLine());
                productoService.crear(nombre, precio, stock, categoriaId);
                System.out.println("¡Producto creado!");
            } catch (NumberFormatException e) {
                System.out.println("Error: precio, stock o ID inválidos.");
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
        case "3" -> {
            try {
                productoService.listar().forEach(p -> System.out.println(p.resumen()));
                System.out.print("Ingrese ID del producto: ");
                Long id = Long.parseLong(scanner.nextLine());
                System.out.print("Nuevo nombre: ");
                String nombre = scanner.nextLine();
                productoService.editar(id, nombre);
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
        case "4" -> {
            try {
                productoService.listar().forEach(p -> System.out.println(p.resumen()));
                System.out.print("Ingrese ID del producto: ");
                Long id = Long.parseLong(scanner.nextLine());
                System.out.print("Ingrese ID de la categoría: ");
                Long categoriaId = Long.parseLong(scanner.nextLine());
                productoService.eliminar(id, categoriaId);
                System.out.println("¡Producto eliminado!");
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
        case "0" -> return;
    }
}
```

TP_Integrador_Prog2 (run) #3

--- SISTEMA DE PEDIDOS (FOOD STORE) ---

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir

Seleccione una opción: 2

--- GESTIÓN DE PRODUCTOS ---

1. Listar
2. Crear
3. Editar
4. Eliminar
0. Salir

```
private void menuUsuarios() throws Exception {
    System.out.println("\n--- GESTIÓN DE USUARIOS ---");
    System.out.println("1. Listar\n2. Crear\n3. Editar\n4. Eliminar\n0. Salir");
    String op = scanner.nextLine();
    switch (op) {
        case "1" -> {
            List<Usuario> lista = usuarioService.listar();
            if (lista.isEmpty()) {
                System.out.println("No hay usuarios registrados.");
            } else {
                System.out.println("--- LISTADO DE USUARIOS ---");
                lista.forEach(System.out::println);
            }
        }
        case "2" -> {
            System.out.print("Nombre: "); String nombre = scanner.nextLine();
            System.out.print("Apellido: "); String apellido = scanner.nextLine();
            System.out.print("Email: "); String mail = scanner.nextLine();
            System.out.print("Celular: "); String cel = scanner.nextLine();
            System.out.print("Contraseña: "); String contra = scanner.nextLine();

            if (nombre.trim().isEmpty() || apellido.trim().isEmpty() || mail.trim().isEmpty() || cel.trim().isEmpty() || contra.trim().isEmpty()) {
                System.out.println("Error: No se puede crear el usuario. Ninguno de los campos puede estar vacío o contener solo espacios.");
                return;
            }

            Rol rolSeleccionado = pedirRol();
            Long nuevoId = (long) usuarioService.listar().size() + 1;
            usuarioService.crear(new Usuario(nuevoId, nombre, apellido, mail, cel, contra, rolSeleccionado));
            System.out.println("Usuario creado con éxito!");
        }
        case "3" -> {
            usuarioService.listar().forEach(c -> System.out.println(c.resumen()));
        }
        case "4" -> {
            // TODO
        }
        case "0" -> {
            return;
        }
    }
}

// TP Integrador_Prog2 (run) #3 X
// 1. requisitos
// 2. Usuarios
// 3. Usuarios
// 4. Pedidos
// 0. Salir
// Seleccione una opción: 3
// --- GESTIÓN DE USUARIOS ---
// 1. Listar
// 2. Crear
// 3. Editar
// 4. Eliminar
// 0. Salir
// 2
// Nombre: 4
// Apellido: dsa
// Email:
```

```

    }
}

private void menuPedidos() throws EntidadNoEncontradaException {
    System.out.println("\n--- GESTIÓN DE PEDIDOS ---");
    System.out.println("1. Listar\n2. Crear\n3. Editar\n4. Eliminar\n0. Salir");
    String op = scanner.nextLine();
    switch (op) {
        case "1" -> pedidoService.listar().forEach(System.out::println);
        case "2" -> {
            try {
                System.out.print("Ingrese ID de Usuario responsable del pedido: ");
                Long uId = Long.parseLong(scanner.nextLine());
                Usuario u = usuarioService.buscarPorId(uId);
                Pedido nuevo = new Pedido((long) pedidoService.listar().size() + 1, Estado.PENDIENTE, FormaPago.EFECTIVO, u);
                // Bucle para agregar varios productos
                boolean agregando = true;
                while (agregando) {
                    System.out.print("ID de producto a agregar: ");
                    Long prodId = Long.parseLong(scanner.nextLine());
                    Producto p = productoService.buscarPorId(prodId);

                    System.out.print("Cantidad: ");
                    int cant = Integer.parseInt(scanner.nextLine());

                    try {
                        // Llamamos al método directamente sobre el objeto 'nuevo'
                        nuevo.addDetallePedido(cant, p);
                        System.out.println("Producto agregado al pedido.");
                    } catch (StockInsuficienteException e) {
                        System.out.println("Error: " + e.getMessage());
                    }

                    System.out.print("¿Agregar otro producto? (s/n): ");
                    agregando = scanner.nextLine().equalsIgnoreCase("s");
                }
            }
        }
    }
}

```

```

TP_Integrador_Prog2 (run) #3 x
2. productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 4

--- GESTIÓN DE PEDIDOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Salir
2
Ingrese ID de Usuario responsable del pedido: 2
ID de producto a agregar: 3

```

```

// --- 3. USUARIOS ---
Usuario admin = new Usuario(1L, "Juan", "Perez", "juan@gmail.com", "+549358355872", "abcd1234", Rol.ADMIN);
Usuario usuario = new Usuario(2L, "Maria", "Gomez", "maria.gomez@hotmail.com", "+549358101224", "wxyz6789", Rol.USUARIO);
//se almacenan
userService.crear(usuario);
userService.crear(admin);

// --- 4. PEDIDOS ---
Pedido pedido1 = new Pedido(1L, Estado.PENDIENTE, FormaPago.TARJETA, admin);
Pedido pedido2 = new Pedido(2L, Estado.CONFIRMADO, FormaPago.EFECTIVO, admin);
Pedido pedido3 = new Pedido(3L, Estado.TERMINADO, FormaPago.TRANSFERENCIA, usuario);
Pedido pedido4 = new Pedido(4L, Estado.PENDIENTE, FormaPago.TARJETA, usuario);

// --- 5. DETALLES Y GUARDADO ---
pedido1.addDetallePedido(2, coca);
pedido1.addDetallePedido(1, agua);
pedido1.addDetallePedido(3, hamburguesa);
pedidoService.crear(pedido1);

```

```

put - TP_Integrador_Prog2 (run) #3 x
2. Crear
3. Editar
4. Eliminar
0. Salir
1
--- LISTADO DE USUARIOS ---
Usuario ID: 2 | Nombre: Maria | Apellido: Gomez | Rol: USUARIO
Usuario ID: 1 | Nombre: Juan | Apellido: Perez | Rol: ADMIN

```

5. Enlaces

- Repositorio en Github: [Link al repositorio](#)
- Video presentación: [Ver video](#)

6. Bibliografía

- Guzman, Analia. "Paradigmas de programación". (2024)
- Apuntes materia paradigmas de programación, UTN FRC 2024.
- Apuntes materia Programación 2, UTN 2026.